

Unit of Work

# Unit of Work Pattern

One transaction, one commit

## DEFINITION

Track objects changed during a business transaction and coordinate writing them out — and concurrency — in one commit.

A Unit of Work watches everything you create, change and delete during one business transaction, then writes them all out in a single commit. It also tracks identity (one in-memory object per row) and coordinates concurrency — so the database moves from one consistent state to the next, never landing half-updated.

## WHY IT MATTERS

Business operations touch many rows across several aggregates. Saving them one statement at a time risks a crash mid-way that leaves inconsistent state, and costs a round-trip per write. One Unit of Work makes the whole operation atomic — all of it commits or none of it does — and flushes efficiently as a single batch.



# — How it works

|              |                                                                                                                                 |
|--------------|---------------------------------------------------------------------------------------------------------------------------------|
| Unit of Work | Tracks new / dirty / removed objects through one business transaction and owns the commit. Usually the ORM session or context.  |
| Repositories | Register their changes with the unit instead of writing immediately — they enlist, the unit commits.                            |
| commit()     | One atomic flush of every tracked change, in dependency order, inside a single DB transaction; rolls all of it back on failure. |

HOW TO APPLY

1. Open a unit at the start of a business transaction (a request, a command).
2. Let repositories register changes on it rather than saving eagerly.
3. Do all the work, then commit once — every tracked change flushes together.
4. On any error roll back the whole unit; keep units short to limit lock time.

LITMUS TEST

If this operation failed halfway, could the database be left inconsistent? If yes, those writes belong inside one Unit of Work.



## — Smell → fix

Two writes, two commits — a crash between them corrupts state:

```
await orders.insert(order) // commit 1
await stock.decrement(item) // commit 2 — may never run
// crash between them: order saved, stock untouched
```

↓ COMMIT AS ONE UNIT

```
await dataSource.transaction(async (em) => {
  em.save(order) // tracked, not yet flushed
  em.save(stock) // tracked
}) // one commit: all changes land, or none do
// TypeORM EntityManager is the Unit of Work
```

Both changes are tracked and flushed in one transaction: if either fails, the whole unit rolls back. In TypeORM this is the EntityManager inside dataSource.transaction; in EF it's DbContext.SaveChanges, in Hibernate the Session.

### USE WHEN

- ✓ Multiple writes must be atomic
- ✓ Reduce round-trips and keep consistency

### AVOID WHEN

- ▲ Single-statement writes
- ▲ Where the ORM session already does it



# — Trade-offs & pitfalls

PROS

- + Atomicity
- + Fewer DB round-trips
- + Consistency

CONS

- Complexity
- Long units risk lock contention

COMMON PITFALLS

- ▲ Long-running units holding locks — contention and deadlocks under load.
- ▲ Sharing one unit/session across concurrent requests instead of one per transaction.
- ▲ Hand-rolling a Unit of Work when the ORM session already is one.

WHERE YOU SEE IT

- ▶ `TypeORM dataSource.transaction / EntityManager; EF Core DbContext.SaveChanges`.
- ▶ `Hibernate / JPA Session` flushing a batch at commit.
- ▶ A per-request transaction wrapping several repository writes atomically.

SEE ALSO

|            |                                                                                                 |
|------------|-------------------------------------------------------------------------------------------------|
| Repository | repositories register their new/dirty/removed objects; the Unit of Work commits them together   |
| Command    | a unit is a transactional command — do-all or undo-all, like an executable change-set           |
| DIP        | the domain enlists changes through an abstraction; the transaction technology stays at the edge |
| Memento    | rollback restores prior state — the same "snapshot and revert" idea at transaction scale        |



## — Interview & sources

### Where have you already used Unit of Work?

Every ORM session implements it: TypeORM transaction/EntityManager, EF DbContext, Hibernate Session all track changes and flush them in one commit.

### Repository vs Unit of Work?

A repository accesses one aggregate type; a Unit of Work spans many and owns the transaction boundary — the commit that makes all the repository changes atomic.

### What does a long Unit of Work cost?

Held locks and contention: the longer the unit stays open, the more it blocks other writers and risks deadlocks. Keep units short — one per business transaction.

### How does Unit of Work relate to the aggregate boundary?

The aggregate is the consistency boundary for one transaction; the Unit of Work is the mechanism that commits one or more aggregate changes atomically. In strict DDD you commit one aggregate per transaction.

#### REMEMBER

Collect every change, then commit once — all of it lands, or none of it does.

#### SOURCES

Martin Fowler — "Patterns of Enterprise Application Architecture" (2002): Unit of Work

TypeORM transactions, EF Core DbContext, Hibernate Session — Unit of Work in practice

Eric Evans — DDD: the aggregate is the consistency / transaction boundary